Blue Origin Interview Study Guide

Electrical Systems Engineering Intern, Summer 2027 · Tuesday Nov 3, 2:00 PM · Chris Ladd and Luca Frix

Built around your portfolio sample (Ranger TVC, the 4-page version you're submitting). Every number here is from your code.

0. How to use this guide

1. **First (30 min):** Section 1 (game plan + walkthrough script) and Section 9 (cheat sheet). Say the walkthrough out loud twice.
2. **Then (60–90 min):** Section 2, your project page by page. These are the questions you're most likely to get, because interviewers ask about what you put in front of them.
3. **Then:** Sections 3–6, the fundamentals. Skim what you know and slow down on what you don't.
4. **Last 20 min before:** Section 7 (weak spots), Section 8 (behavioral), and the cheat sheet again.

The one rule

If you don't know something, say "I haven't worked with that directly, but here's how I'd reason about it," then reason out loud. The recruiter told you they want to see your thought process. A confident wrong answer is the worst outcome; an honest "I'd check X, and here's why" is fine.

1. Game plan and walkthrough script

Likely structure (50 min)

Time	What happens	Your goal
0–5 min	Intros, "tell us about yourself"	45-second pitch (Section 8)
5–15 min	You walk through the portfolio	5–7 minutes, then stop and invite questions
15–40 min	Technical Q&A: your project, then fundamentals	Explain the why behind each decision
40–50 min	Behavioral, why Blue, your questions	Two or three good questions ready

Portfolio walkthrough script (~6 minutes)

Share your screen with the PDF. Point at things as you talk. Don't read the page; they can read.

Page 1 (1.5 min). *"Ranger is our lab's VTVL liquid rocket. To steer it, the engine sits on a two-axis gimbal pushed by two electric linear actuators. I own the TVC system from the flight computer to those actuators. [Point at Fig. 1.] The flight computer is a Teensy 4.1 running Zephyr. The main controller runs on a 1.5 ms tick. My TVC thread runs at 50 Hz and is the only thing that talks to the CAN-FD bus. On the bus are two moteus motor controllers, one per axis. Each runs its own fast current and position loops at tens of kHz, so my job is to send a position setpoint with limits every 20 ms, read back the state, and decide whether the actuators are allowed to move at all. [Point at requirements.] I designed to five requirements, and I'll show how each one is verified at the end."*

Page 2 (2 min). *"This page is the electrical interface. Each actuator needs a 43-byte message per cycle. That's one CAN-FD frame versus six classic CAN frames, and since CAN is differential it handles the noise near a firing engine. The key physical-layer decisions: a 3.3 V transceiver because the Teensy pins aren't 5 V tolerant; 120 Ω termination at both ends because the moteus boards don't have it; twisted pair with a ground. [Point at timing bar.] At 1 Mbps a 48-byte frame takes about half a millisecond, so four frames per cycle is about 2 ms of a 20 ms cycle, well inside my 5 ms reply window. For bring-up I went layer by layer: scope on CAN_H and CAN_L, a 60 Ω check for termination, SocketCAN to see frames independently of my firmware, then one actuator at a time."*

Page 3 (1.5 min). *"This is the control chain. A gimbal angle command gets NaN-checked, clamped to ±12°, slew limited to 30°/s, converted to an actuator length with a law-of-cosines model from our mechanical team, converted to motor revolutions through the 4 mm lead screw, and sent to the moteus with velocity, acceleration and torque limits plus a 100 ms watchdog. [Point at state machine.] The supervisor won't enable unless both controllers reply, both are homed and centered, and the config is complete. Five fault types latch it into FAULT, which centers healthy axes, stops the faulted one, and aborts the test."*

Page 4 (1 min). *"The timing design: an absolute-period kernel timer so the loop doesn't drift, priority placement so sensors can't delay actuator commands, and a minimal mutex hand-off so the controller never waits on CAN. Each requirement maps to a test. The two problems I'm proudest of are a static-initialization-order bug I caught before bench testing and the threading redesign. Next steps are checking signal quality on the real harness before turning on the 5 Mbps data rate, setting a torque limit, and hardware-in-the-loop testing. Happy to go deeper on any of it."*

2. Your project, page by page

For each claim on your portfolio: what it means, why you made that choice, and how to answer the follow-ups.

2.1 System overview (page 1)

Know cold

- **Flight computer:** Teensy 4.1 = NXP i.MX RT1062, ARM Cortex-M7 at 600 MHz, 1 MB RAM. Runs Zephyr RTOS. Code in C++.
- **Why FlexCAN3:** the RT1062 has three CAN controllers; only CAN3 supports CAN-FD. On the Teensy 4.1 it's on pins 30 (RX) and 31 (TX).
- **Actuators:** BLDC motor directly driving a 4 mm lead screw. 1 rev = 4 mm, so 25.4 / 4 = 6.35 rev per inch.
- **Motor controllers:** moteus r4.1 (open-source BLDC controller by mjbots). Does field-oriented control (FOC) and closes current, velocity and position loops internally at tens of kHz. You talk to it over CAN-FD with a register-based protocol.
- **Ground link:** Ethernet, protobuf messages (nanopb on the Teensy). Python ground client with a live TVC dashboard and CSV logging.

> **Q: Walk me through what happens in one 20 ms cycle.**
> The kernel timer fires. The TVC thread copies the latest pitch/yaw command from the controller (under the mutex). It runs the supervisor's step on the replies from last cycle: checks for faults, advances the state machine, and decides what to send each axis (query, stop, position command, or set-home). It sends one frame per axis, every frame requesting a reply. It waits up to 5 ms for both replies, parses them, records latency, and publishes telemetry back to the controller. Then it blocks on the timer until the next period. CPU time is tens of microseconds; the rest is waiting.

> **Q: Why does the moteus close the position loop instead of your firmware?**
> The motor needs current and position control at kHz rates to be stable and stiff. I couldn't do that over a 50 Hz bus. The moteus does commutation and the tight loops locally; I send setpoints and limits. It's a standard split: fast inner loop near the hardware, slower outer loop in the flight computer. It also means a comms dropout doesn't leave the motor uncontrolled; it keeps holding its last command until its watchdog trips.

> **Q: Why 50 Hz? Isn't that slow for TVC?**
> 50 Hz is the setpoint rate, not the actuator's control bandwidth; the moteus runs its loops far faster. The engine gimbal is a large mass moving at most 30°/s, so the mechanical bandwidth is low. 50 Hz with under 5 ms of latency was enough margin for the GNC rate we need. If GNC needed more, the bus has room: about 10% load now, and the 5 Mbps data phase would cut frame time by about 3×. *Be careful not to claim a GNC bandwidth number you don't know.*

> **Q: What exactly did you do versus your teammates?**
> Use the page-1 lists. Mine: CAN-FD bring-up, the moteus protocol and transport, the TVC thread, kinematics in firmware, the supervisor, bus debugging. Teammates: the controller framework, sensor drivers, the first moteus bench test (which set the controller IDs), and the gimbal mechanism and MATLAB model. Being precise here builds trust.

2.2 Requirements R1–R5

Be ready for "where did these requirements come from?" Honest answer: they're the requirements I wrote for the TVC software from what the vehicle needed (gimbal range from the mechanical design, the loop rate from GNC, and the safety ones from thinking through how it could hurt someone or the engine). Systems engineers love hearing that you wrote requirements before code and traced each to a verification.

Verification methods (the four Blue/NASA use)
Test, **A**nalysis, **I**nspection, **D**emonstration. Your R1 uses test (unit tests + bench sweep). R2 is analysis (the timing budget) plus test (live telemetry). R3–R5 are test (unit tests). If asked "how would you verify R4 on the vehicle?": pull the CAN connector during a powered bench run and confirm the axis faults within 60 ms and the moteus stops within 100 ms.

2.3 CAN-FD choice and physical layer (page 2)

Q: Why CAN-FD over classic CAN, RS-485, Ethernet, or PWM?
Classic CAN: 8 data bytes per frame; my 43-byte message would need 6 frames per axis, and moteus speaks CAN-FD anyway. **RS-485/422:** also differential and robust, but no built-in arbitration, error detection/retransmission or acknowledgement; I'd have to build a protocol. **Ethernet:** more bandwidth than needed, more complex, needs switches/magnetics, and it's not what the moteus supports. **PWM:** single-ended, one wire per actuator, no feedback, no error detection. CAN-FD gives a differential multi-drop bus, hardware CRC and ACK, automatic retransmission, and priority by ID.

Q: Why 120 Ω, and why at both ends?
A twisted pair has a characteristic impedance of roughly 120 Ω. When a signal edge travels down the line and hits an end whose impedance doesn't match, part of it reflects back. Terminating each end with a resistor equal to the line impedance absorbs the edge. Both ends, because the signal can travel either way: any node can transmit. The moteus boards have no termination on board, so I added it. **Check:** with power off, measure across CAN_H and CAN_L: two 120 Ω in parallel = 60 Ω. 120 Ω means one is missing; ~40 Ω means there's an extra one.

Q: What happens if you forget termination?
Reflections cause ringing on the edges. At low bit rates on a short bench cable it may "work"; at higher rates or longer harnesses the ringing lands on the sample point and you get bit errors, error frames, and eventually nodes going error-passive or bus-off. Also, without any termination the recessive state isn't actively pulled together (the termination is what discharges the bus back to recessive), so edges get slow. This is exactly why you verify signal quality on the real harness before raising the data rate.

Q: What do CAN_H and CAN_L look like on a scope?
Recessive (logic 1): both about 2.5 V, differential ≈ 0 V. **Dominant (logic 0):** CAN_H ≈ 3.5 V, CAN_L ≈ 1.5 V, differential ≈ 2 V. Receivers decide on the difference: above ~0.9 V is dominant, below ~0.5 V is recessive. Use two probes (or a differential probe) and the math channel H − L. Look for: clean levels, ringing or overshoot on edges (termination), slow edges (capacitance, missing termination), and a dominant ACK bit near the end of each frame (some node received it).

Q: Why twisted pair, and why also run a ground?
Twisting makes both wires pick up nearly the same induced noise (same loop area alternating sign), so it's common-mode and the differential receiver rejects it. It also reduces emissions. The ground keeps both transceivers' common-mode voltages within the allowed range (roughly −2 V to +7 V for standard ISO 11898-2 transceivers). Without it, a ground offset between the boards (from motor currents in the power return, for example) can push the bus outside that range.

Q: Why a 3.3 V transceiver?
The i.MX RT1062 I/O runs at 3.3 V and isn't 5 V tolerant. A transceiver with 5 V logic output on RXD would drive 5 V into an MCU pin and could damage it. Many transceivers have a separate VIO pin to set the logic level; the bus side is still the standard CAN levels.

> **Q: What's the standby pin and why does it matter?**
> Many CAN transceivers have a standby or silent pin. In standby the transmitter is off (low power, only wake-up detection). If it floats, it may sit in standby, so the node can't transmit or even ACK. Tie it to the level for normal mode (or drive it from a GPIO).

2.4 CAN fundamentals you should know

- **Arbitration:** the bus is wired-AND: dominant (0) overrides recessive (1). Every node transmits its ID bit by bit while listening; if it sends recessive but reads dominant, it lost and backs off. Lowest ID wins, and no data is destroyed ("non-destructive bitwise arbitration"). That's why IDs set priority.
- **Bit stuffing:** after 5 identical bits in a row, the transmitter inserts an opposite bit, so receivers get edges to resynchronize to. That's why frame length varies a bit with the data ("+ stuffing" in your equation).
- **ACK:** any receiver that got a valid frame drives the ACK slot dominant. A transmitter with no ACK keeps retrying, which is why your transmit is non-blocking with a callback: a missing node won't stall the thread.
- **Error handling:** CRC, stuff, form, bit and ACK errors. Each node has transmit and receive error counters. Above 127 it goes *error passive*; transmit counter above 255 means *bus-off* (it stops transmitting until recovery). This stops one broken node from jamming the bus.
- **Bit timing:** each bit is divided into time quanta: sync, propagation, phase 1, phase 2. The sample point is usually 75–87.5% of the bit. All nodes must agree on bit rate and roughly on sample point.
- **Bus length:** during arbitration a bit has to reach the far end and back within one bit time, so at 1 Mbps the bus is limited to roughly 40 m. Propagation is about 5 ns/m.
- **CAN-FD differences:** payload up to 64 bytes (lengths 0–8, 12, 16, 20, 24, 32, 48, 64; that's why 43 bytes is padded to 48); the **BRS** (bit rate switch) bit lets the data phase run faster (your 5 Mbps) while arbitration stays at 1 Mbps so all nodes can arbitrate; stronger CRC (17 or 21 bits) and a stuff-bit counter. At high data rates transceivers need delay compensation, which is one reason you validate on the real harness first.
- **Extended ID:** 29 bits. moteus packs a prefix, the source ID, the destination ID and a "reply requested" bit into it. Your hardware filter matches all 29 bits so each controller's replies land in their own queue.

2.5 Frame-time math

One bit at 1 Mbps = 1 µs. A 48-byte CAN-FD frame is about 67 bits in the arbitration phase (start bit, 29-bit ID and control bits, ACK, end of frame, inter-frame space) plus about 430 bits in the data phase (384 data bits + CRC + stuff bits + DLC). At 1 Mbps everywhere: ~500 µs ≈ 0.5 ms. With BRS at 5 Mbps: 67 µs + 430 × 0.2 µs ≈ 153 µs ≈ 0.15 ms.

> **Q: How did you get 10% bus load?**
> Four frames per cycle (two commands, two replies) × 0.5 ms = 2 ms per 20 ms = 10%. CAN buses are usually kept under about 50–70% load so lower-priority traffic still gets through and there's room for retransmissions.

> **Q: Why float32 instead of int16?**
> moteus can encode registers as int8, int16, int32 or float. The int16 position format has a resolution of 0.0001 rev, so it tops out at ±3.2767 rev. Over ±12° the actuator moves about −8.1 to +7.0 rev, so int16 would overflow. float32 costs a few bytes per register but removes the problem; the frame still fits easily in 48 bytes.

2.6 Firmware ↔ hardware interface (page 2)

> **Q: What's a devicetree?**
> A data structure (from Linux, also used by Zephyr) that describes the board's hardware separately from the code: which peripherals exist, which pins they use, clock and bit-rate settings. Zephyr turns it into C macros at build time. My overlay enabled FlexCAN3, set the bit rates (1 Mbps / 5 Mbps), and selected it as the board's CAN bus. Kconfig is the build-time feature switch system (enable CAN, enable CAN-FD mode). I added a compile-time `#error` if CAN-FD isn't configured, so a wrong build can't reach the bench.

> **Q: What do hardware acceptance filters do?**
> The CAN controller compares incoming IDs to filters in hardware and only interrupts the CPU for matches. I add one filter per moteus, matching all 29 bits, and route each to its own message queue. That keeps unrelated traffic away from the CPU and means the pitch reply can never be confused with the yaw reply.

> **Q: What if the bus is stuck or a node is unplugged?**
> Transmit is non-blocking: `can_send` with `K_NO_WAIT` and a completion callback, so the frame goes into a hardware mailbox and the thread continues. If nobody ACKs, the controller keeps retrying in hardware, but my thread isn't waiting on it. The real liveness check is the reply: no reply for 3 cycles = fault. Then the moteus's own 100 ms watchdog stops the motor if commands stop.

2.7 Control chain (page 3)

Stage	What it does	Why
Reject NaN/inf	A non-finite command holds the previous angle	Upstream math bugs (divide by zero) produce NaN; NaN passes through clamps and converting NaN to an integer is undefined behavior in C++
Clamp ±12°	Hard limit on gimbal angle	Mechanical range; protects the engine, feed lines and structure
Slew limit 30°/s	Command moves toward the target by at most 30°/s × 20 ms = 0.6° per cycle	Prevents violent steps that load the structure or excite resonances; conservative placeholder to be tuned
Inverse kinematics	Angle → actuator length (law of cosines), then a length clamp	The actuator controls length, not angle
Length → rev	(L − L_center) × 6.35 × direction sign	moteus works in output revolutions; homing makes center = 0 rev
moteus command	Position + velocity limit (1.5× slew), accel limit (300°/s² equivalent), torque limit, 100 ms watchdog	moteus' own trajectory limiter can never lag my slew-limited setpoint; watchdog is the comms-loss backstop

2.8 The kinematics

Picture a triangle with corners at the gimbal pivot, the actuator's mount on the structure, and the actuator's mount on the engine. Two sides are fixed lengths (pivot to structure mount, L_{ts}; pivot to engine mount, L_{re}). The third side is the actuator, L. The angle between the fixed sides is θ_0 at center, and the gimbal angle δ adds to it. Law of cosines:

$$L^2 = L_{ts}^2 + L_{re}^2 - 2 \cdot L_{ts} \cdot L_{re} \cdot \cos(\theta_0 + \delta)$$

- **Inverse direction** (length → angle, for telemetry and fault checks): $\cos(\theta_0+\delta) = (L_{ts}^2 + L_{re}^2 - L^2) / (2 \cdot L_{ts} \cdot L_{re})$, then $\delta = \text{acos}(\ldots) - \theta_0$. The code clamps the acos argument to $[-1, 1]$ so a noisy measurement can't produce NaN.

- **Why asymmetric travel** (−8.1 vs +7.0 rev): L depends on $\cos(\theta_0+\delta)$, which isn't linear in δ, so equal angle steps give unequal length steps.
- **Sensitivity:** $dL/d\delta \le 0.115$ in/deg over ±12°. I use the maximum to convert an angle-rate limit into an actuator-speed limit conservatively: 30°/s × 0.115 ≈ 3.45 in/s ≈ 22 motor rev/s.
- **Validation:** unit tests compare the firmware to 241 points per axis (−12° to +12° in 0.1° steps) generated from the MATLAB model, plus tests that length → angle inverts angle → length and that length increases monotonically with angle.
- **Precomputed:** $L_{ts}^2 + L_{re}^2$ and $2 \cdot L_{ts} \cdot L_{re}$ are constants computed at compile time.

> **Q: Why closed form instead of a lookup table?**
> It's exact, tiny (a sqrt and a cos), invertible analytically, and there's no table to keep in sync with the geometry. A Cortex-M7 with a hardware FPU does it in well under a microsecond. A lookup table would make sense if the geometry had no closed-form solution.

2.9 Lead screw and motor physics (likely if they're electrical-leaning)

- **Lead** = linear travel per revolution = 4 mm (single-start, so lead = pitch).
- **Your actuator uses a lead screw** (sliding threads, not recirculating balls). Typical efficiency is about 20–50% for a plain lead screw versus ~90% for a ball screw.
- **Force from torque:** $F = 2\pi \cdot \eta \cdot T / \text{lead}$. Example: T = 1 N·m, lead = 0.004 m, η ≈ 0.3: $F \approx 2\pi \times 0.3 \times 1 / 0.004 \approx 470$ N (about 105 lbf). The motor torque you need is $T = F \cdot \text{lead} / (2\pi \cdot \eta)$, so low efficiency means more current for the same force.
- **Why a lead screw (tradeoffs):** cheaper, simpler, compact, quiet, and it tends to resist being back-driven. Costs: lower efficiency (more motor current and heat), friction and wear over cycles, and some backlash (play when the direction reverses), which shows up as a small position error at reversals.
- **Backdriving:** whether the engine's load can push the screw backward depends on the lead angle versus the friction angle. A small lead angle with high friction is self-locking: with the motor unpowered, the gimbal tends to stay where it is. That matters for your fault policy: "stop" (unpowered) on a self-locking screw roughly holds position rather than letting the engine flop. **Don't claim it's self-locking unless you've tested it.** The honest answer is "lead screws often are; I'd verify by loading it with power off."
- **Regen is smaller** with a self-locking or low-efficiency screw, because less of the load's energy makes it back to the motor. Still worth bus capacitance for motor deceleration.
- **Torque ∝ current** in a BLDC: $T = K_t \cdot I$. So "torque limit" really means "phase current limit".
- **Back-EMF:** a spinning motor generates voltage ∝ speed (K_e). At high speed it limits how fast you can go for a given supply voltage.
- **Regeneration:** when the load pushes the motor (decelerating, or the engine pushing back), the motor acts as a generator and can pump current back into the supply, raising the bus voltage. Supplies need capacitance or a brake resistor to absorb it. Good thing to mention if they ask about actuator power.
- **FOC** (field-oriented control): the controller measures phase currents and rotor angle and controls the current vector so it's always 90° to the rotor's magnetic field, which gives maximum torque per amp and smooth torque.

2.10 Safety supervisor (page 3)

State	What it does
STARTUP	Queries both controllers until both reply. Never drives.
CLEARING	Sends stop to both to clear any old fault, until both report stopped.
READY	Checks enable conditions every cycle; reports which ones block (shown on the dashboard).
HOMING	Operator has centered the gimbal with a jig. Sends "set output exact = 0" so moteus treats this position as zero, then waits for the home state to read back. Times out after 25 cycles (0.5 s).
ENABLED	Sends position commands. Checks for faults every cycle.
FAULT	Latched. Healthy axes slew to center, faulted ones are stopped, active sequence aborts. Cleared by re-homing.

> **Q: Why do you need homing?**
> The moteus encoder measures motor angle, but after power-up it doesn't know where gimbal center is in absolute terms across multiple revolutions. Homing defines the current position as zero while the gimbal is mechanically centered. The supervisor refuses to enable until moteus reports it has been homed since power-up AND the measured length is within 0.05 in of center. A Teensy reboot alone doesn't need re-homing as long as the moteus stayed powered and is still near center.

> **Q: Why latch faults instead of auto-recovering?**
> A fault means something unexpected happened. Auto-recovering could hide a real problem and let the system oscillate between working and failing. Latching forces a person to look, then re-home, which also re-establishes the position reference.

> **Q: Why 3 missed cycles, and why 5 cycles for tracking error?**
> One missed reply can be a transient (a corrupted frame that got retried late). Three in a row (60 ms) is a real loss, and it's still faster than the moteus' 100 ms watchdog, so the flight computer notices first. The tracking-error persistence filter (5 cycles = 100 ms) keeps normal lag during a fast move from tripping it. Both are tunable, and the tracking limit (0.10 in) is marked to tune from measured response.

> **Q: "Wrong mode" fault?**
> If the moteus isn't in position mode while we think it's enabled (it reset, faulted, or timed out), it's not following commands. It gets a 3-cycle grace period, then it faults.

Likely hard question: "Is centering the gimbal actually safe in flight?"
Good answer: "On the test stand, yes: centering points thrust straight down the stand's axis, and the test aborts. In flight it's a vehicle-level decision. Centering a gimbal mid-flight could be worse than holding the last angle, so the right response depends on flight phase, and the real answer might be engine shutdown or a redundant actuator. My supervisor makes the policy configurable (hold center or stop), and the abort is signaled to the controller, so GNC can own the flight decision." This shows you understand fail-safe vs fail-operational.

2.11 Real-time design (page 4)

> **Q: Explain absolute-period scheduling vs sleeping.**
> `while(1) { work(); sleep(20ms); }` has a period of 20 ms + work time + scheduling delay, so it drifts and the frequency isn't exactly 50 Hz. A periodic kernel timer fires every 20 ms regardless of how long the work took; the thread blocks on the timer status (`k_timer_status_sync`), which also returns how many periods expired. More than 1 means you missed a cycle, which counts as an overrun instead of silently stretching the period.

Q: Explain your thread priorities.

In Zephyr a lower number is higher priority, and negative priorities are *cooperative*: they aren't preempted by other threads, only by interrupts, until they block or yield. The controller work queue is −10 (highest), TVC is −7, sensor threads are −5. So when both are ready, the controller tick always runs first, and a slow sensor read can never delay a TVC cycle. Because TVC spends almost all its time blocked on the timer or reply queues, its CPU use is tiny.

Q: What's priority inversion and how did you avoid it?

A high-priority thread waits on a mutex held by a low-priority thread, and a medium-priority thread preempts the low one, so the high one waits indefinitely (the classic Mars Pathfinder bug). Zephyr mutexes do priority inheritance (the holder is temporarily boosted), but I avoided it by design: the shared data is one small struct, both sides only copy it while holding the lock, and nothing blocks, sends on CAN or logs inside the lock. I moved a log call out of the critical section because immediate-mode logging could stall the controller.

Q: What's jitter and how do you measure it?

Jitter is variation in the actual period around the nominal 20 ms. Each cycle I timestamp the wake-up with the CPU cycle counter, compute the period, and track the max deviation over a 1 s window. Also tracked: per-axis reply latency, max cycle time, overruns, missed replies and rejected transmits, logged every 10 s and sent in telemetry.

Be ready: "What jitter did you actually measure?"

If you have a real number from the bench, say it. If you don't, say: "The instrumentation reports it every cycle; I don't have a number I'd quote with confidence right now. The design budget is that both replies come back within the 5 ms window, and overruns are counted, not hidden." Never invent a number.

2.12 The two problems (page 4)

Static initialization order: explain it like I'm an EE.

In C++, global objects in different source files are constructed before `main()`, but the order between files isn't defined. A geometry constant (2·L_ts·L_re) was a global computed at startup. The supervisor was also a global whose constructor used that constant to precompute the length limits. If the supervisor's file happened to initialize first, the constant was still 0, the min and max lengths collapsed to the same value, and every command would be clamped to one length. Unit tests passed because there the order happened to be right. Fix: make the constant `constexpr`, computed by the compiler (I wrote a compile-time Newton's-method square root because `std::sqrt` isn't constexpr), plus a `static_assert` that its value is in the expected range. Now it can't be wrong at runtime, and a geometry change that breaks it fails the build.

The threading problem: what was racing?

Waiting for CAN replies (up to 5 ms) inside the 1.5 ms controller tick would stall every control task. Moving CAN to its own thread created shared state (latest command in, latest status out) touched by two threads. The fix: one thread owns the bus; the hand-off is a single struct behind a mutex, copied in and out; nothing slow happens under the lock. The NaN fix: the bench log converted a float to an integer for printing, and converting NaN (e.g. a value that wasn't available that cycle) to an int is undefined behavior in C++, so I check for NaN first. *Know which specific symptom you saw; if they ask "how did you notice the race?", answer from your memory, not this guide.*

3. Electrical fundamentals

Electrical systems roles at Blue cover avionics integration, power distribution, wire harnesses, and test. Expect a few of these, often as "how would you approach..." questions.

3.1 Circuit basics (answer instantly)

Concept	What to know
Ohm's law, power	$V = IR$. $P = VI = I^2R = V^2/R$.
KVL / KCL	Voltages around a loop sum to zero; currents into a node sum to zero.
Series / parallel R	Series add. Parallel: $1/R = \Sigma 1/R_i$; two equal resistors in parallel = R/2 (your 60 Ω).
Voltage divider	$V_{out} = V_{in} \cdot R2/(R1+R2)$. Loading by the next stage changes it.
RC circuit	$\tau = RC$; 63% after τ, ~99% after 5τ. Low-pass cutoff $f_c = 1/(2\pi RC)$.
RL circuit	$\tau = L/R$. An inductor resists changes in current: $V = L \cdot di/dt$. Interrupting current makes a large voltage spike.
Capacitor	$I = C \cdot dV/dt$. Blocks DC, passes AC. Impedance $1/(2\pi fC)$ falls with frequency.
Thevenin / Norton	Any linear network = a voltage source with a series resistance (or current source with parallel R).
Impedance matching	Max power transfer at $R_{load} = R_{source}$. For transmission lines: terminate in Z_0 to stop reflections.

3.2 Components

- **Diode:** conducts one way, ~0.7 V drop (silicon), ~0.3 V (Schottky). Uses: reverse-polarity protection, rectification, flyback.
- **Flyback diode:** across an inductive load (solenoid, relay coil, motor). When the switch opens, the inductor's current continues through the diode instead of creating a huge voltage spike that kills the switch. Downside: slow turn-off. A Zener or TVS clamp turns off faster.
- **TVS diode:** clamps transient overvoltage (ESD, load dump, spikes) by conducting hard above its breakdown voltage. Put them on connector pins that leave the box, like CAN lines.
- **MOSFET as a switch:** N-channel low-side switch (between load and ground) is simplest: gate high = on. Key specs: $R_{DS(on)}$ (conduction loss I^2R), $V_{GS(th)}$ (need enough gate voltage to fully turn on, not just the threshold), V_{DS} rating, gate charge (switching speed). High-side switching (between supply and load) usually uses P-channel or an N-channel with a gate driver/charge pump. High-side is preferred when you want the load grounded when off.
- **BJT vs MOSFET:** BJT is current-controlled (base current), MOSFET is voltage-controlled (gate is a capacitor, ~zero DC current). MOSFETs dominate power switching.
- **Op-amp:** ideal rules: no current into the inputs; with negative feedback, the inputs sit at the same voltage. Non-inverting gain = 1 + Rf/Rg; inverting gain = −Rf/Rin. Uses: amplifying sensor signals, buffering, filtering, differential/instrumentation amps.
- **Pull-up/pull-down resistors:** set a defined default level on an input that would otherwise float (like your transceiver standby pin). Required on open-drain buses like I^2C.
- **Decoupling capacitors:** small ceramic caps (e.g. 100 nF) right at each IC power pin supply fast current spikes locally, so the switching current doesn't travel through the power trace inductance. Plus bulk caps for slower demand.

3.3 Power

- **Linear regulator (LDO):** simple and quiet, but wastes $(V_{in} - V_{out}) \times I$ as heat. 12 V → 3.3 V at 1 A wastes 8.7 W. Efficiency ≈ V_{out}/V_{in}.
- **Switching converters:** buck (step down), boost (step up), buck-boost. 85–95% efficient, but produce switching noise, so they need filtering and layout care.
- **Inrush current:** big input capacitors look like a short at power-on. Limit it with a soft-start, NTC thermistor, or hot-swap controller.
- **Protection:** fuses (one-time) or resettable PTCs/eFuses for overcurrent; a series diode or ideal-diode MOSFET for reverse polarity; TVS for transients; undervoltage lockout so electronics don't brown out unpredictably.
- **Power budget:** list every load's voltage, nominal and peak current, and duty cycle; sum per rail; add margin (commonly 20–50%); check battery capacity (Ah) and wire/connector ratings against peaks. Motor stall current is the peak to design for.
- **Batteries:** capacity in Ah, energy in Wh = V × Ah. Li-ion cell ~3.7 V nominal, 4.2 V full, ~3.0 V empty. C-rate: 1C draws the full capacity in 1 hour.

Know your actuator power before 2:00
They may ask "how are the actuators powered?" Find out the supply voltage, what battery or supply, the moteus input current at stall, and whether it's fused. If you don't know: "That's owned by our electrical lead; the moteus reports bus voltage in my telemetry, and here's how I'd size the supply…" (then walk through a power budget). Mention regen: a backdriven motor pumps energy back into the bus.

3.4 Wire harnesses and connectors (core electrical systems topic)

- **Wire sizing:** pick gauge for (1) current without overheating (with derating for bundles, altitude and temperature: wires in a bundle can carry less than in free air), and (2) acceptable voltage drop: $\Delta V = I \times R_{wire} \times$ length (both directions!). Rough numbers: 22 AWG ≈ 53 mΩ/m, 20 AWG ≈ 33 mΩ/m, 18 AWG ≈ 21 mΩ/m, 16 AWG ≈ 13 mΩ/m. Every 3 AWG steps is about 2× the area.
- **Worked example:** 10 A over 2 m of 18 AWG (out and back = 4 m): R ≈ 4 × 0.021 = 0.084 Ω; ΔV ≈ 0.84 V; loss ≈ 8.4 W. On a 24 V bus that's 3.5% drop. Fine for a motor, maybe not for a sensor reference.
- **Twisting:** twist signal pairs and power/return pairs together. That minimizes the loop area, which reduces both emitted and picked-up magnetic noise.
- **Shielding:** a braided or foil shield blocks electric-field coupling. Low-frequency rule: ground the shield at one end to avoid ground loops. At high frequency, ground at both ends (360° at the connector backshell). Many aerospace harnesses do both ends.
- **Connectors:** aerospace uses circular MIL-spec (e.g. MIL-DTL-38999) with locking, keying, sealing and backshells. Derate pin current, use keying so mating connectors can't be swapped, and don't put power and sensitive signals on adjacent pins.
- **Crimp vs solder:** crimps are preferred in flight hardware: solder wicks into the strands and makes a stiff point that cracks under vibration.
- **Strain relief, service loops, bend radius, labeling,** and keeping the harness away from heat and sharp edges.
- **Testing a harness:** continuity (each pin goes where the drawing says), isolation/insulation resistance (no shorts between pins or to shield, often a "hipot" or megger test at high voltage), and checking against the wiring diagram/pinout (the ICD).

3.5 Grounding and EMI

- **Ground loop:** two grounded points at slightly different potentials connected by more than one path. Current flows in the loop and shows up as noise on signals that use ground as reference. Fix: single-point grounding, differential signals,

isolation.

- **Return current:** every current returns to its source. At high frequency the return follows the path of least *inductance*, right under the signal trace. Breaking the return path (split ground plane) creates big loops that radiate.
- **Separate noisy and quiet:** keep motor/power currents from flowing through the ground reference of sensitive analog circuits. Star ground or separate returns that join at one point.
- **Differential signaling** (CAN, RS-422, LVDS, Ethernet) rejects common-mode noise; that's your page-2 argument.
- **Coupling types:** capacitive (electric field, high dV/dt; fix with distance or shielding), inductive (magnetic, high dI/dt; fix by reducing loop area and twisting), conducted (through shared power or ground; fix with filtering).

3.6 Sensors and signal acquisition

- **ADC:** resolution N bits → step (LSB) = $V_{ref}/2^N$. 12-bit at 3.3 V ≈ 0.8 mV. SAR ADCs are common in microcontrollers; sigma-delta for high resolution at low speed.
- **Nyquist:** sample at more than twice the highest frequency in the signal, or higher frequencies alias (fold) into the band you care about. Put an analog low-pass (anti-aliasing) filter before the ADC.
- **Pressure transducers:** often 0.5–4.5 V ratiometric or 4–20 mA. A 4–20 mA current loop ignores wire resistance and voltage drop, and 0 mA means a broken wire, so you can detect the fault.
- **Thermocouples:** two metals produce a small voltage (~40 µV/°C for type K) proportional to the temperature *difference* between the hot junction and where the wires terminate, so you need cold-junction compensation. Tiny signal, so amplify and filter.
- **Strain gauge / Wheatstone bridge:** four resistors; the gauge's tiny resistance change unbalances the bridge, giving a small differential voltage you amplify with an instrumentation amp. Used in load cells (thrust measurement).
- **Encoders:** quadrature (A/B channels 90° apart, gives direction) or absolute. The moteus uses a magnetic encoder on the motor.

3.7 Digital interfaces

Bus	Wires	Notes
UART	TX, RX (+GND)	Asynchronous, agreed baud rate, point to point, single-ended. RS-232/422/485 are physical layers for it; RS-422/485 are differential.
SPI	SCLK, MOSI, MISO, CS per device	Synchronous, full duplex, fast (MHz–tens of MHz), short distances on a board. Master drives the clock.
I^2C	SDA, SCL	Two wires, open-drain with pull-ups, addressed, slower (100 k–1 MHz), on-board. Pull-up value trades speed (rise time) against power.
CAN / CAN-FD	CAN_H, CAN_L (+GND)	Differential, multi-drop, arbitration, CRC, ACK; automotive and many vehicles.
Ethernet	Differential pairs + magnetics	High bandwidth, switched; transformer-isolated. Your ground link.
MIL-STD-1553	Differential, transformer-coupled	Classic aerospace command/response bus, 1 Mbps, dual-redundant. Worth knowing it exists.

4. Embedded software and RTOS concepts

- **RTOS vs bare metal:** an RTOS gives threads with priorities, preemption, timers, and synchronization (mutexes, semaphores, queues), with predictable timing. Bare metal is a super-loop plus interrupts.
- **Deterministic** means the worst-case timing is bounded and known, not just the average. Avoid unbounded loops, dynamic memory, blocking calls with no timeout, and logging in hot paths.
- **Mutex vs semaphore:** a mutex is ownership (lock/unlock by the same thread, supports priority inheritance); a semaphore is a counter used for signaling between threads or ISRs. You used a mutex for shared state and a timer/message queue for signaling.
- **Message queue:** thread-safe FIFO; your CAN receive callback (ISR context) puts frames in a queue, and the thread takes them with a timeout. That's how you wait "up to 5 ms" for replies.
- **ISRs:** keep them short; no blocking. Hand work to a thread.
- **Why allocation-free:** malloc/new in real time is non-deterministic (variable time) and fragments memory over time. Your moteus encoder/decoder uses fixed 64-byte buffers, no heap.
- **volatile vs atomic:** `volatile` stops the compiler from caching a variable (for hardware registers), but doesn't make read-modify-write safe between threads. Use atomics or a mutex for that. Your transmit-error counters are atomics because the ISR callback increments them.
- **Undefined behavior:** e.g. converting NaN or an out-of-range float to an int. The compiler may do anything. Check before converting.
- **std::expected:** C++23 type holding either a value or an error, used instead of exceptions (often disabled on embedded).
- **Watchdog timers:** hardware that resets the system (or, for the moteus, stops the motor) if software stops "petting" it. Catches hangs.
- **Unit tests on native_sim:** Zephyr can build for the host PC, so hardware-independent logic runs as a normal program. That's why you kept the supervisor, kinematics and protocol free of CAN/driver dependencies.

5. Controls, briefly

- **PID:** P reacts to the current error, I removes steady-state error (but can wind up while saturated), D damps by reacting to the rate of change (but amplifies noise, so it's usually filtered). Anti-windup: stop integrating when the output is saturated.
- **Cascaded loops:** inner loop much faster than the outer loop (rule of thumb 5–10×). Yours: moteus current → velocity → position at kHz rates, inside your 50 Hz setpoint loop, inside GNC.
- **Latency hurts stability:** a delay adds phase lag that grows with frequency and eats phase margin. That's why you keep reply latency under 5 ms and bounded.
- **Saturation and rate limits** are nonlinearities: the controller has to know about them or they cause windup and overshoot.
- **TVC physics:** pitching moment = $T \cdot d \cdot \sin\delta$, where T is thrust, d is the distance from the gimbal to the vehicle's center of mass, and δ is the gimbal angle. Lateral force = $T \sin\delta$; you lose $T(1 - \cos\delta)$ of axial thrust (about 2% at 12°).

6. Systems engineering and safety

- **Requirements:** verifiable ("shall respond within 5 ms"), not vague ("shall be fast"). Each has a verification method (TAID) and traces up to a higher-level need and down to design. You did this on a small scale with R1–R5.
- **Interface control document (ICD):** defines every signal between subsystems: pinout, voltage levels, protocol, timing. Your page-2 interface table is a mini ICD.

- **FMEA** (failure modes and effects analysis): for each component, list how it can fail, the effect, how you'd detect it, and the mitigation. Example for your system: *CAN wire breaks → no replies → detected in 60 ms → FAULT, gimbal to center, abort; moteus watchdog stops the motor at 100 ms.*
- **Single points of failure:** in your design, the CAN bus is shared by both actuators, so a short on CAN_H/CAN_L takes out both axes. A flight design might use separate buses or a redundant bus. Saying this unprompted shows maturity.
- **Fail-safe vs fail-operational:** fail-safe = go to a safe state (stop) when something breaks; fine for a test stand. Fail-operational = keep working through a failure (needs redundancy); often required for crewed vehicles in critical phases.
- **Test like you fly:** test with flight harnesses, flight software builds and realistic conditions. Your "verify signal quality on the real vehicle harness before raising the bit rate" is an example.
- **Hardware-in-the-loop (HIL):** real flight computer and/or actuators connected to a simulation of the rest of the vehicle. Your next step.

7. Your weak spots and honest answers

Interviewers probe the edges of what's on the page. Prepare these so they don't surprise you.

> **"Has this flown or been hot-fired?"**
> Be precise and honest: it's been developed and tested in unit tests and on the bench with a single actuator. Vehicle integration and hot fire are next. Say what's pending: the 5 Mbps data rate on the real harness, an explicit torque limit, tuning from measured step response, HIL.

> **"Your background looks software-heavy. Why electrical systems?"**
> "My degree is ECE and my coursework includes linear circuits and embedded systems, but my most interesting problems on this project were at the hardware boundary: voltage levels, termination, timing on the wire, what a motor controller does on its own. I want to go deeper into the electrical side: power, harnessing, avionics integration and test. That's why this role." Your resume also lists Altium, KiCad and PCB design. Have a specific example ready if they ask.

> **"What are the numbers from your bench test?"**
> Only quote numbers you actually measured. Otherwise describe what was measured and how, and what the design limits are.

> **"Why did you pick ±12°, 30°/s, 0.10 in, 0.05 in?"**
> ±12° is the mechanical gimbal range from the mechanical team. 30°/s, the 300°/s² acceleration limit and the 0.10 in tracking limit are conservative starting values to be tuned on hardware. 0.05 in homing tolerance is small relative to the 2.4 in stroke (about half a degree), tight enough to catch a wrong home but loose enough for normal settling. Saying "placeholder, here's how I'd tune it" is a good answer.

> **"What would you do differently?"**
> Design the power side alongside the software: a power budget for the actuators, current limits and regen handling from day one. Get the real harness on the bench earlier. Consider separate CAN buses per axis or redundancy for flight.

> **Code they might ask you to sketch**
> Be able to write a slew limiter: `step = max_rate * dt; out = cur + clamp(target - cur, -step, step);` and a clamp that rejects NaN: `if (!std::isfinite(x)) return last; return std::clamp(x, -lim, lim);`

8. Behavioral

Elevator pitch (45 seconds)

"I'm Adit, a junior in Electrical and Computer Engineering at USC. On the USC Liquid Propulsion Lab's GNC team I own the thrust vector control system for Ranger, our VTVL rocket: the CAN-FD bus to the motor controllers, the real-time control thread, and the safety logic that decides when the engine gimbal can move. This past summer I was a network engineering intern at EchoStar, where I automated configuration intake and cut turnaround from days to under 15 minutes. And I do research on UAV path planning at USC. What ties it together is that I like making systems that touch real hardware work reliably. I want to spend next summer doing that on flight hardware at Blue."

Why Blue Origin

- The mission: a future where millions of people live and work in space for the benefit of Earth. Say what that means to you personally.
- **Gradatim Ferociter** ("step by step, ferociously"): your project was built that way, from unit tests to single-actuator bench builds to the full system, each step before the next.

- Reusability and the vehicles: New Shepard (suborbital, BE-3PM engine), New Glenn (orbital, seven BE-4 LOX/methane engines on the reusable first stage), BE-4 also powering ULA's Vulcan, and Blue Moon landers for Artemis. Mention one you actually find exciting. **Read the latest Blue Origin news this morning** so you can mention something current; don't rely on this guide for recent events.
- Your interest is specifically in electrical systems on vehicles like these: power, avionics, harnesses, and their verification.
- Read the Leadership Principles page linked in your email and pick two to tie to your stories.

STAR stories to have ready

Prompt	Story
Technical challenge	CAN-FD bring-up / threading redesign (page 4)
Finding a hidden problem	Static-init-order bug caught in review before hardware
Impact / efficiency	EchoStar: config intake from days to under 15 minutes, 95%+ less manual entry, 1,000+ parameters
Leadership	Beta Theta Pi VP: 60+ members, $5,000+ raised
Working with another team	Turning the mechanical team's MATLAB model into firmware and validating against it
Failure / what you'd change	Pick a real one. Close with what you changed afterward.

Format: Situation (1–2 lines), Task (your part), Action (what you did, tools, tradeoffs), Result (a number) + what you'd do next time. Keep each to about 2 minutes.

Questions to ask them

- What does the electrical systems team own day to day on your program: harness, power, avionics integration, test?
- What did a strong intern's project look like last summer?
- How does your team balance moving fast with flight-hardware rigor?
- What's a recent problem your team solved that you're proud of?

9. One-page cheat sheet

Item	Value
Flight computer	Teensy 4.1, i.MX RT1062 Cortex-M7 600 MHz, Zephyr RTOS, C++
CAN controller / pins	FlexCAN3 (only CAN-FD controller on the RT1062), pin 31 TX, pin 30 RX
Bit rates	1 Mbps arbitration, 5 Mbps data (BRS currently off: 1 Mbps everywhere)
Message size	43 bytes → padded to 48 (CAN-FD sizes: 12, 16, 20, 24, 32, 48, 64)
Frame time	~0.5 ms at 1 Mbps; ~0.15 ms with 5 Mbps data
Bus traffic	4 frames/cycle ≈ 2 ms of 20 ms ≈ 10% load
Loop	50 Hz (20 ms), absolute-period kernel timer, 5 ms reply window
Priorities	Controller −10, TVC −7, sensors −5 (lower = higher, negative = cooperative)
CAN levels	Recessive: both ~2.5 V, diff ≈ 0. Dominant: H ≈ 3.5, L ≈ 1.5, diff ≈ 2 V
Termination	120 Ω each end; 60 Ω across H–L with power off
Logic level	3.3 V; RT1062 pins not 5 V tolerant
Gimbal	±12°, 2 axes, ~2.4 in stroke (17.08 → 18.36 → 19.47 in)
Lead screw	Lead screw (not ball screw), 4 mm lead, 1:1 direct drive → 6.35 rev/in; ±12° ≈ −8.1 / +7.0 rev
Why float32	int16 position saturates at ±3.2767 rev
Kinematics	$L^2 = L_ts^2 + L_re^2 - 2 \cdot L_ts \cdot L_re \cdot \cos(\theta_0+\delta)$; $dL/d\delta \le 0.115$ in/deg
Limits	Slew 30°/s (0.6°/cycle); moteus velocity limit 1.5× slew; accel 300°/s²
Enable conditions	Both reply, no faults, homed since power-up, within 0.05 in of center, config complete
Faults (5)	No reply (3 cycles / 60 ms), moteus fault, watchdog timeout, wrong mode (3-cycle grace), tracking > 0.10 in for 5 cycles
Fault response	Latched; healthy axes to center; faulted axes stopped; sequence aborts; clear by re-homing
moteus watchdog	100 ms
Homing	Jig at center → "set output exact = 0" → verify home state; 0.5 s timeout
Tests	55 unit tests (29 supervisor, 11 kinematics, 15 moteus protocol); 241 MATLAB points per axis; ±2° bench sweep at 0.5°/s
Static-init bug	Global constant used in another global's constructor; fix: constexpr + static_assert
Next steps	5 Mbps on real harness, explicit torque limit, tune limits from step response, HIL

Fundamentals to say without thinking

$V = IR$ · $P = I^2R$ · $\tau = RC$ · $f_c = 1/(2\pi RC)$ · $V = L \cdot di/dt$ · Nyquist: $f_s > 2f_{max}$ · LSB = $V_{ref}/2^N$ · lead-screw force $F = 2\pi\eta T/\text{lead}$ · BLDC torque $T = K_t \cdot I$ · CAN: lowest ID wins, 5-bit stuffing, bus-off at TEC > 255 · twisted pair = smaller loop area · single-point ground avoids ground loops · flyback diode on inductive loads.

Mindset

Talk through your reasoning. Say "I don't know, here's how I'd find out" when true. Tie answers back to reliability, safety and verification. End answers; don't ramble. Have fun with it: you built something real.